Context-aware conversational module

Conversation user/agent (one-to-one) | *text only*

- Answer question with a LLM, remembering previous interaction

Question
str: Natural language
Ex: "What should I eat tonight?"

History
> memory of all previous interactions
dict: JSON file

```
{
  "user1": [
    {
      "role": "user",
      "content": "J'aime la nourriture italienne"
    },
    {
      "role": "assistant",
      "content": "C'est un excellent choix ! Les plats italiens sont délicieux et variés."
    }
  ],
  "user2": [
    {
      "role": "user",
      "content": "Je suis allergique à la tomate"
    },
    {
      "role": "assistant",
      "content": "Je suis désolé d'apprendre cela."
    }

  (...)
```

User
int: Unique ID for the user
Ex: "1" is associated to "user1"

Select the history corresponding to the given user, and provide the LLM (gpt4o) with the 10 last interactions (as context)

Function "ask_llm"

```
def ask_llm(question, history, user):
```

Evaluation

Golden Answer
str: Natural language

Answer
str: Natural language
Ex: "You should eat pizza"

Score*

***Scores, metrics:**

- NLP metrics on raw text: *ROUGE, METEOR...*
- Embedding-based metrics: *cosine-similarity...*
- Execution time: *average time required to generate 1 answer*
- "Recall accuracy": *An information is provided in the history (ex. : birthday). We directly **ask** for this information (users' question) and see if the answer **contain** the information (ground truth)*

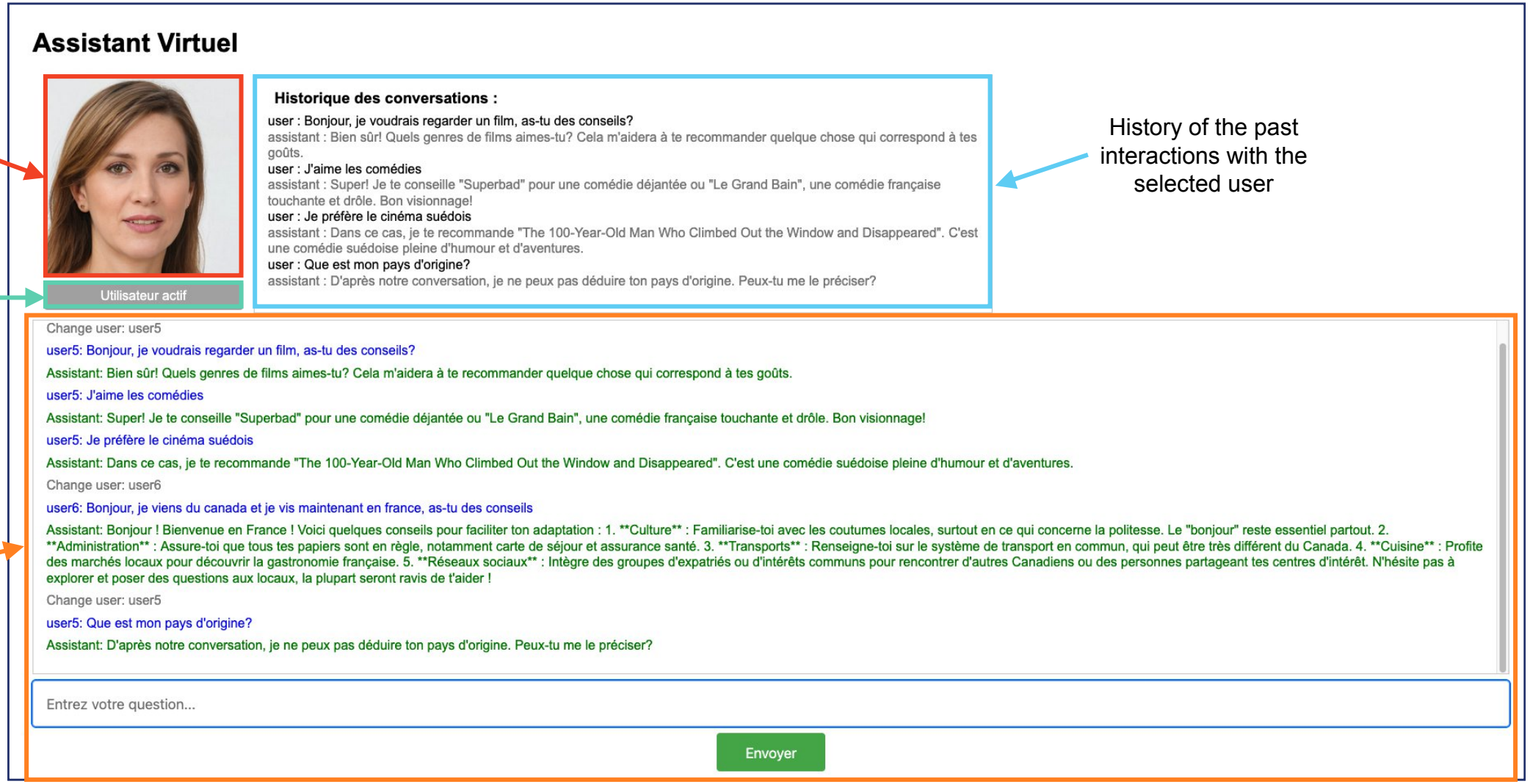
User interface

Image of the selected user (image embedding associated to user id)

Button allowing to change current user (choosing a new picture)

Chat-box: Send question and visualize assistant's answers

History of the past interactions with the selected user



What next?

- Issue with history:
  - Only the last *n* sentences considered by the LLM (*n: hard-coded parameter)*
    ➡ If an information was given in the *(n+1)*^{e} last interaction, it will not be known by the LLM
    (Limit *n* is required to mitigate computational cost, time).

  ➢ Use interactions to build a **user profile** for each user *(name, birthday, what they like/don't like...)*
  ➡ Provide a *compact context* about the current user to the LLM (and the last interactions to be able to resume a conversation)

- User image should simulate image flow of the camera
  - Different images for the same user (*with different expressions, posture, background*), to test the ability to recognize faces
  - Images with many users visible
    ➡ Add a module to focus on the user speaking to the camera

Long-term memory

```
"user_memory": [
        {
            "name": "nom",
            "description": "Martha"
        },
        {
            "name": "preferences",
            "description": "Martha préfère être appelée
par son prénomet souhaite qu'on se tutoie."
        },
        {
            "name": "hobbys",
            "description": "Martha aime jouer au tennis
et lire des livres policiers."
        },
        {
            "name": "emploi",
            "description": "Martha travaille comme
professeure de français depuis 16 ans"
        }
    ]
```

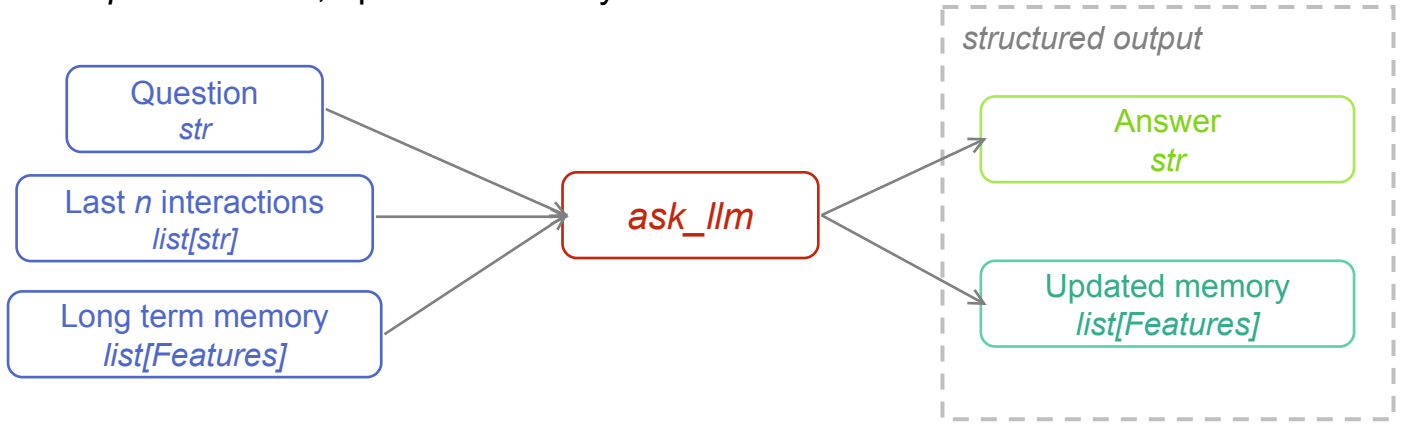
Long term memory implemented as a list of user-related features

- Updated at each interaction:
  Single prompt, *structured output*:

```
class AnswerWithMemory(BaseModel):
    answer: str
    updated_memory: list[Feature]
```

```
class Feature(BaseModel):
    name: str = Field(..., pattern=r"^\w+$", description="Doit être un mot
unique sans espaces ni caractères spéciaux, décrivant la catégorie d'une
caractéristique utilisateur (par exemple: nom, âge, hobby...).")
    description: str
```

- *Input:* User's question ; n last interactions (arbitrary number) ; long term memory
  Output: Answer ; updated memory



- A lot of features in long term memory = lot of tokens (input and output)
  ➥ Latency in dialog !!